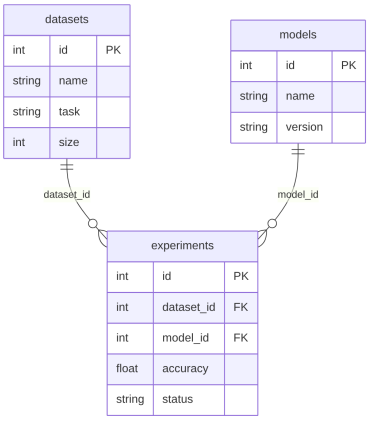
MiniDB: отчёт по лабораторной работе №1

Чапурин Никита, ТИДИ25 · файловая мини-СУБД на C++17

MiniDB хранит данные в собственных файлах без SQLite, pandas и ORM. Реализованы таблицы с типами INT / FLOAT / STRING и первичным ключом, INSERT / SELECT / UPDATE / DELETE, консольный интерфейс, **внешние ключи с проверкой ссылочной целостности**, JOIN, backup и индекс **id → offset**. Данные сохраняются после перезапуска.

1. Схема данных (dbdiagram)



Три таблицы: **datasets**(id, name, task, size), **models**(id, name, version) и **experiments**(id, dataset_id, model_id, accuracy, status). PK каждой таблицы **id** (int). Типы: int (64 бита), float (double), string (до 64 байт).

Внешние ключи **experiments.dataset_id → datasets.id** и **experiments.model_id → models.id** объявляются при создании таблицы и записываются в каталог. Движок проверяет их при каждой операции: нельзя вставить или изменить эксперимент с несуществующим dataset_id / model_id, нельзя удалить датасет или модель, на которые есть ссылки, нельзя удалить таблицу, на которую ссылаются. DBML для dbdiagram.io лежит в schema.dbml.

2. Формат хранения

Директория **minidb_data/** содержит **catalog.txt** (метаданные, по строке на таблицу) и по файлу **<таблица>.dat**. Каталог пишется во временный файл и переименовывается поверх старого. Строка каталога (внешние ключи и первичный ключ в конце):

```
experiments;id:int;dataset_id:int;model_id:int;accuracy:float;status:text;fk=dataset_id>datasets.id;fk=model_id>models.id;pk=id
```

Файл данных это записи **фиксированной длины** подряд, без заголовка. Запись начинается с байта-флага (1 = жива, 0 = удалена), далее значения столбцов. Пример для experiments (97 байт на запись; datasets 145 байт, models 137):

Поле	Тип	Размер, байт	Смещение в записи
флаг (жива / удалена)	byte	1	0
id (PK)	int	8	1
dataset_id (FK)	int	8	9
model_id (FK)	int	8	17
accuracy	float	8	25
status	string	64, добивается нулями	33
Итого		**97**	

Запись с номером k лежит по смещению **k × 97**, поэтому UPDATE пишет на прежнее место. **Индекс id → offset** это хеш-таблица в памяти (значение PK → смещение). Отдельного файла индекса нет: при открытии таблицы он строится проходом по .dat и не может разойтись с данными. Условие «id = число» читает запись одним seek, остальные условия просматривают файл целиком. DELETE ставит флаг 0 и убирает ключ из индекса, compact физически убирает такие записи. Проверка внешнего ключа это поиск по PK родительской таблицы, то есть тоже через индекс.

3. Примеры операций (консоль, тестовые данные: 5 datasets, 5 models, 20 experiments)

Команда	Результат
`insert into experiments values (21, 1, 1, 0.93, completed)`	1 row inserted; запись в конец файла, индекс 21 → смещение
`select id, status, accuracy from experiments where accuracy > 0.9`	выбор столбцов + фильтр: 7 строк, например (4, completed, 0.99)
`select * from experiments where id = 21`	1 строка; поиск через индекс
`update experiments set status = completed, accuracy = 0.8 where id = 22`	1 row(s) updated; запись перезаписана на месте
`delete from experiments where id = 21`	1 row(s) deleted; флаг = 0, ключ убран из индекса

`insert into experiments values (23, 99, 1, 0.5, failed)`	**error: foreign key violation: dataset_id = 99 does not exist in datasets.id**; запись не добавлена
`delete from datasets where id = 1`	**error: cannot delete datasets.id = 1: referenced by experiments.dataset_id**
`select * from experiments join datasets on experiments.dataset_id = datasets.id`	20 строк, 9 столбцов (поля обеих таблиц)
`backup minidb_backup`	копия каталога и .dat-файлов; открывается как отдельная база
`перезапуск программы`	все таблицы и изменения на месте (show tables: 5 / 5 / 20)

4. Benchmark индекса

Сравниваются два запроса, возвращающие одну строку: **id = k** (через индекс, «после») и **code = k** (столбец без индекса, полный проход, «до»). На размер: 10 000 запросов по индексу и 100 по полному проходу, случайные ключи. Таблица bench(id PK, code, name), отдельная от рабочей базы.

Строк	Файл	INSERT, мс (всего)	Открытие + индекс, мс	Без индекса, мкс/запрос	С индексом, мкс/запрос	Ускорение
1 000	79 КБ	1,4	0,1	38,0	1,7	×23
10 000	791 КБ	18,9	1,1	400,3	1,8	×226
50 000	3955 КБ	71,6	6,1	2184,6	2,5	×870

Время без индекса растёт линейно с числом строк, с индексом остаётся почти постоянным. Условия: Linux x86_64, 2 ядра, g++ 13.3, -O2. Между запусками значения менялись в 1,5–2 раза (ускорение на 50 000 строк: ×870 и ×906), порядок величин сохраняется; абсолютные числа зависят от машины.

5. Ограничения

Нет журналирования и fsync (данные сбрасываются после каждой операции, но при отключении питания не гарантированы); один процесс; индекс только по PK; в WHERE одно условие (AND / OR не реализованы); JOIN вложенными циклами O(n·m); PK только int и не изменяется; строка до 64 байт.